ExamGuard Proctoring System

1. Project Overview

ExamGuard is a local desktop-based exam proctoring prototype that monitors a student through a webcam during an online examination. It runs fully on the user machine, avoiding cloud video upload and improving privacy. The system detects face presence, multiple faces, looking-away behavior, and suspicious motion, then records important session evidence in local logs and reports.

2. Objectives

- Build a real-time webcam monitoring tool for online exams.
- Detect no-face, multiple-face, gaze, and motion-related violations.
- Reduce false alerts from small movements and brief detection glitches.
- Keep the interface clean and distraction-free for the student/proctor.
- Store CSV logs, screenshots, and summaries locally for review.

3. System Architecture

The application starts from main.py and creates a Tkinter dashboard. The dashboard coordinates camera capture, face detection, motion detection, alert scoring, UI updates, and report generation. Processing runs on a background thread so the interface remains responsive.

```
main.py
  -> ui.dashboard.ProctoringDashboard
      -> core.camera.CameraManager
      -> core.face_detector.FaceDetector
      -> core.motion_detector.MotionDetector
      -> core.alert_engine.AlertEngine
      -> ui.report_generator.ReportGenerator
```

4. Runtime Pipeline

Input	Webcam frame captured through OpenCV and resized to the configured resolution.
Detection	MediaPipe BlazeFace detects faces; OpenCV MOG2 detects meaningful motion.
Evaluation	The alert engine confirms violations across sustained frames and applies cooldowns.
Output	The dashboard shows compact status indicators while logs and summaries are saved locally.

5. Core Technologies

Python is used as the main language. Tkinter provides the desktop interface. OpenCV handles camera access and motion detection. MediaPipe provides the primary face detection model. NumPy and Pillow support frame processing and display. fpdf2 and CSV logging support reporting workflows.

6. Main Components

CameraManager opens the webcam, captures frames on a background thread, and returns safe frame copies. FaceDetector uses MediaPipe BlazeFace as the primary detector and Haar cascades as a fallback. MotionDetector uses background subtraction to classify movement by zone and magnitude. AlertEngine converts detections into confirmed violations using thresholds, cooldowns, score changes, and reset support. ReportGenerator writes local CSV logs, screenshots, and session summaries.

7. Recent Improvements

- Replaced Haar-only detection with MediaPipe BlazeFace for stronger face detection.
- Added a local model file: models/blaze_face_short_range.tflite.
- Kept Haar cascades as a fallback when MediaPipe is unavailable.
- Reduced false motion alerts by increasing motion thresholds.
- Added sustained-frame confirmation before alerts are fired.
- Removed distracting face boxes, motion boxes, alert log panel, and visible risk score card.
- Verified Reset Score clears score, cooldowns, and confirmation streaks.

8. Testing and Validation

The project includes a full system test suite in test_system.py. It validates imports, dashboard structure, motion detection, face detection, alert behavior, report generation, integrated pipeline behavior, and camera lifecycle. The latest verified result is:

```
PASSED : 39
FAILED : 0
TOTAL  : 39
All tests passed!
```

9. Conclusion

ExamGuard demonstrates a practical local proctoring workflow using computer vision and rule-based analysis. The system captures webcam frames, detects faces and motion, evaluates suspicious activity, and stores evidence locally. The latest updates make the application more reliable and user-friendly by integrating MediaPipe face detection, reducing false motion alerts, simplifying the interface, and improving reset/report behavior.

Overall, the project provides a strong foundation for a privacy-preserving exam monitoring tool. Its modular design allows future detection and reporting features to be added without rewriting the core application workflow.